Ollscoil Teicneolaíochta an Oirdheiscirt
South East Technological University

AI PRINCIPLES AND PRACTICE · WEEK 7

How a Machine Learns, and How It Is Wired

Reference booklet: guess, check, blame, nudge

Plain English first. Every formula written three ways: in symbols, in words, and with numbers you can check on your phone.

Lecturer: Indrakshi

How to use this booklet

This booklet goes with the Week 7 lecture and lab. It follows the lecture's seven parts in the same order, with the same examples, and slows each one down: every picture is explained, every formula is written out three ways, and every number is worked through so you can check it yourself.

You need no maths beyond add, subtract and multiply. When a formula appears, it appears in a box like the one below: first in symbols (the way books write it), then the same formula in words, then the same formula with our example's numbers plugged in. Read the middle line first.

THE FORMULA	$\hat{y} = ax + b$
IN WORDS	**price = a × size + base price**
WITH NUMBERS	**€240,000 = 2,000 × 90 m² + €60,000**

★ KEEP THIS
A red box holds something you need for the lab, the final project and the Week 12 viva. There is one at the end of every part.

GOING FURTHER · OPTIONAL ·
A grey box is optional. Read it if you are curious; skip it freely.

Your time: about 45 minutes for Parts 1 to 7. Part 8 (going further) is optional. The practice questions, cheat sheet and glossary at the back are for revision.

Where the numbers come from. One small network, trained on two interlocking crescents of dots (the "two moons"), produces every picture and number. The figures come from `week07-make_figures.py`; the lab programs print the same numbers on your own machine.

Contents

How to use this booklet . . . 2
1 The one picture: guess, check, blame, nudge . . . 5
2 What “learning” means . . . 6
2.1 A model is a formula with dials . . . 6
2.2 Three settings of one dial . . . 6
2.3 Parameters and hyperparameters . . . 7
3 Guess, with one neuron . . . 8
3.1 A tiny decision-maker . . . 8
3.2 A second example: a loan . . . 8
3.3 The S-curve: turning a score into “how sure” . . . 9
3.4 One neuron draws one straight line . . . 9
4 Guess, with a network: how it is wired . . . 11
4.1 Four dots one neuron cannot sort . . . 11
4.2 ReLU: the bend used inside networks . . . 11
4.3 Wiring two hidden neurons . . . 12
4.4 The hidden layer moves the dots . . . 12
4.5 Why every neuron needs a bend . . . 13
4.6 From four dots to a real network . . . 13
4.7 Asking the network a question: the forward pass . . . 14
5 Check: how wrong was it? . . . 16
5.1 For number answers: the average squared miss . . . 16
5.2 For yes/no answers: cross-entropy . . . 17
5.3 Memorising is not learning . . . 17
6 Nudge: walking downhill . . . 19
6.1 Feel the slope, take a step . . . 19
6.2 The formula of the day . . . 19
6.3 The house dial finds its own way down . . . 19
6.4 The step size is the whole ballgame . . . 20
7 Blame: passing it backwards . . . 21
7.1 One dial, worked by hand . . . 21
7.2 Blame by nudging . . . 21
7.3 The chain rule: blame multiplies, like gears . . . 22
7.4 Backpropagation: walk back from the loss . . . 22
7.5 In a real network: three rules . . . 23
7.6 Checking the homework . . . 23
8 The loop, running . . . 24
8.1 The loop as four formulas . . . 24
8.2 A spoonful, not the whole pot . . . 24
8.3 Watching it learn . . . 24
8.4 Three cures for memorising . . . 25
8.5 Reading the curves like a doctor . . . 25
9 Going further (optional) . . . 27
9.1 Smarter steps: momentum and Adam . . . 27
9.2 Big steps first, small steps later . . . 27
9.3 The telephone game: when blame fades . . . 27
9.4 Two tricks that let networks grow very deep . . . 28
9.5 Enough bends can draw any curve . . . 28

9.6 A short history: why did this wait until 2012? 28
10 In the lab 29
Practice questions 30
Formula cheat sheet 32
Glossary 33

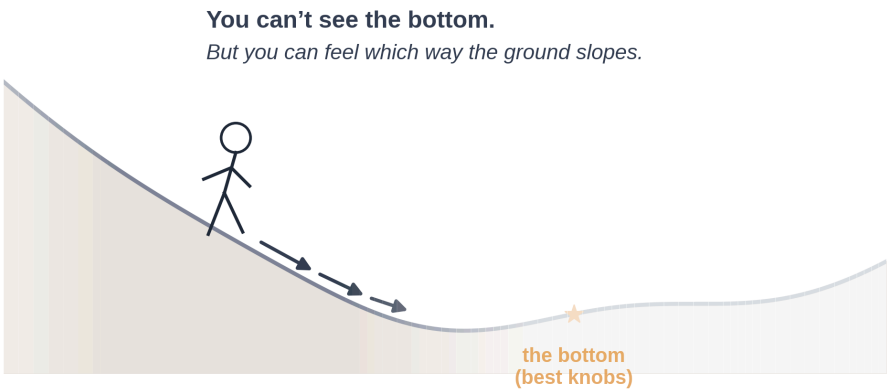
Part 1 The one picture: guess, check, blame, nudge

The whole of this week fits in one sentence:

A machine learns by guessing, checking, and nudging its numbers, thousands of times.

Those four verbs are a loop that every model in this module runs, from Week 2′s straight line to Week 10′s transformer:

Step	Technical name	What happens
1 Guess	forward pass	the model makes a prediction with the numbers it has now
2 Check	loss	one number says how wrong the prediction was
3 Blame	backward pass (backpropagation)	work out which numbers caused the mistake, and how much
4 Nudge	update (gradient descent)	change every number a little, so the next guess is less wrong



The one picture to keep in your head: a hillwalker in fog.

EVERYDAY ANALOGY

You are on a hillside in thick fog and want to reach the valley floor. You cannot see it. But you can feel the slope under your feet, so you take a small step downhill, feel again, and step again.

Every new word this week is a fact about that walk. **Height** is how wrong the machine is. **Where you stand** is the machine’s numbers. **A step** is one small change to those numbers. **The valley floor** is where the machine is least wrong. When you feel lost, come back to the walker.

Part 2 What “learning” means

2.1 A model is a formula with dials

Suppose we want to guess a house’s price from its size. A simple guess is “so many euro per square metre, plus a base price”:

THE FORMULA	$\hat{y} = ax + b$
IN WORDS	**price = a × size + base price**
WITH NUMBERS	**€240,000 = 2,000 × 90 m² + €60,000**

x	what we know: the size of the house
$\hat{y}$ (“y-hat”)	the model’s guess: its price. The hat always means “a guess”; plain y will mean the true answer
a, b	the **dials**: euro per square metre, and the base price

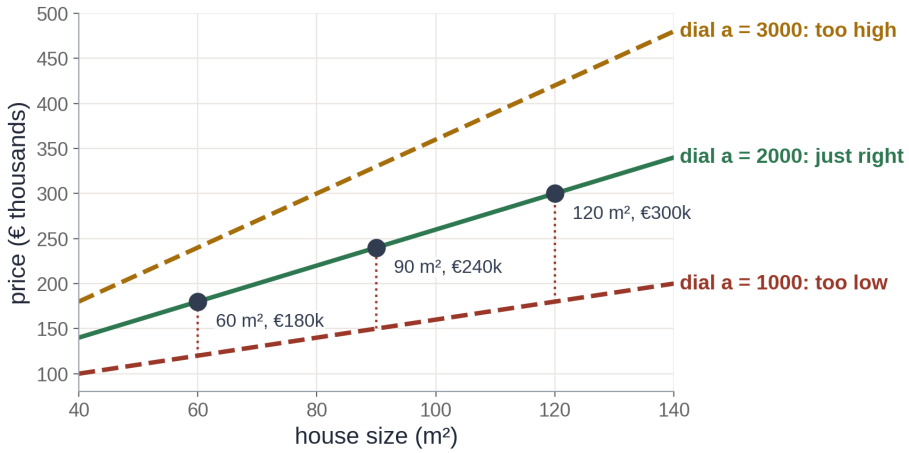
The formula is the **model**. The letters a and b are its **dials**, also called **parameters**. Set the dials badly and every guess is wrong; set them well and the guesses are close.

IN PLAIN ENGLISH

Learning means turning the dials until the guesses are close to the real answers. The computer does the turning. The rest of this booklet is about how.

2.2 Three settings of one dial

We know three real sales: a 60 m² house sold for €180k, a 90 m² house for €240k, a 120 m² house for €300k. Keep the base price at b = €60k and try three values of a:



> **Worked example: three settings.** With $a = 1000$ the guesses are €120k, €150k and €180k: every one too low, missing by 60, 90 and 120 thousand. With $a = 3000$ they are €240k, €330k and €420k: every one too high. With $a = 2000$ they are €180k, €240k and €300k: exactly right.

Trying settings by hand worked here because there is one dial and three houses. Today’s network has 337 dials, so we need the computer to find good settings by itself. Part 6 shows how.

2.3 Parameters and hyperparameters

PARAMETERS

The dials **the computer** turns: a and b here, 337 in today's network, billions in a chatbot.

HYPERPARAMETERS

The settings **you** choose before training starts: how big a step to take, how many layers, how long to train.

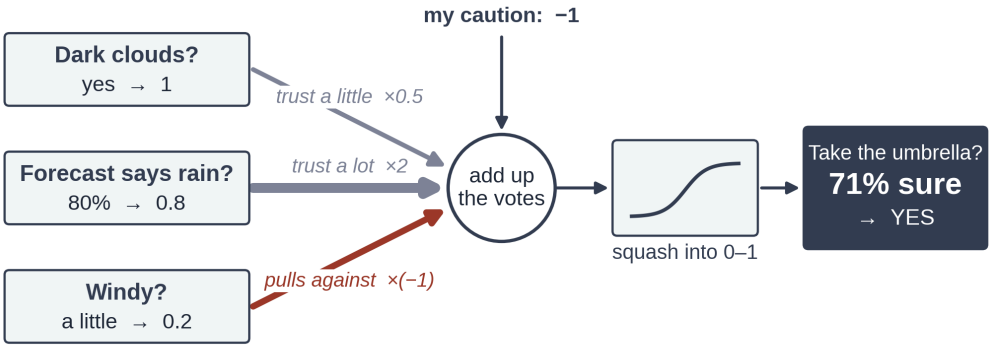
★ KEEP THIS

- A model is a formula with **dials** (parameters). Learning = turning the dials so the guesses match the answers.
- The training loop has four steps: **guess** → **check** → **blame** → **nudge**, repeated many times.
- **Parameters** are learned by the computer. **Hyperparameters** are chosen by you.

Part 3 Guess, with one neuron

Before we wire a whole network, meet its smallest part. A **neuron** is the house formula with a decision bolted on.

3.1 A tiny decision-maker



Every morning you decide whether to take an umbrella. You look at clues: are there dark clouds (yes = 1), what does the forecast say (80% = 0.8), is it windy (a little = 0.2)? You do not trust every clue equally. The forecast counts a lot (×2), the clouds a little (×0.5), and the wind actually argues **against** taking an umbrella (×−1), because it turns umbrellas inside out. Each clue times its trust is that clue's **vote**. You add the votes, add your general caution (−1: you dislike carrying an umbrella), and squash the total into "how sure", between 0 and 1.

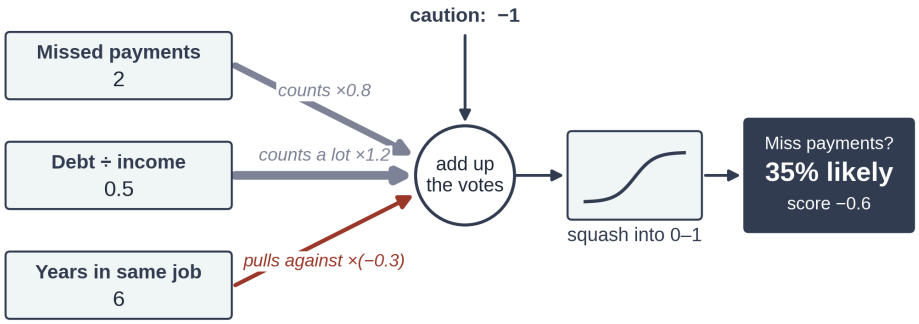
THE FORMULA	$z = w_1x_1 + w_2x_2 + w_3x_3 + b \qquad \hat{y} = \sigma(z)$
IN WORDS	**score = trust × clouds + trust × forecast + trust × wind + caution** **how sure = squash(score)**
WITH NUMBERS	**0.9 = 0.5 × 1 + 2 × 0.8 + (−1) × 0.2 + (−1)** **0.71 = squash(0.9)**

x_1, x_2, x_3	the **inputs**: the clues, as numbers
w_1, w_2, w_3	the **weights**: how much each clue counts. A negative weight means "this clue argues against"
b	the **bias**: a nudge up or down, whatever the clues say
z	the **score**: the votes added up, plus the bias
σ ("sigma")	the **activation**: the squash that turns the score into "how sure"

> **Worked example: the umbrella, by hand.** 1.0 × 0.5 = 0.5; 0.8 × 2.0 = 1.6; 0.2 × (−1.0) = −0.2. The votes add to 1.9; adding the caution −1.0 gives a score of 0.9. Squashed, 0.9 becomes 0.71: **71% sure, take the umbrella.** If your caution were −3 instead, the score would be −1.1 and the answer about 25%: leave it at home.

3.2 A second example: a loan

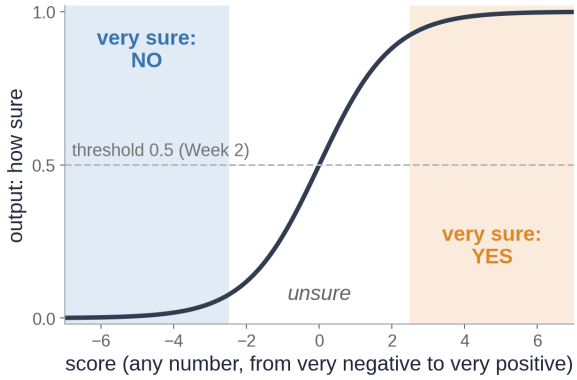
The same neuron can judge a loan, using clues from the credit data we have used since Week 1.



> **Worked example: the loan.** Two missed payments × 0.8 = 1.6. Debt-to-income 0.5 × 1.2 = 0.6. Six years in the same job × (−0.3) = −1.8: a long, steady job pulls the score **down**. The votes add to 0.4; with the caution −1 the score is −0.6, which squashes to **35% likely to miss payments**.

These trust values are made up to keep the arithmetic easy. In a real model they are **learned** from past customers, and Week 3's question still applies: is any clue quietly standing in for a protected group?

3.3 The S-curve: turning a score into "how sure"



THE FORMULA $\sigma(z) = \frac{1}{1+e^{-z}}$

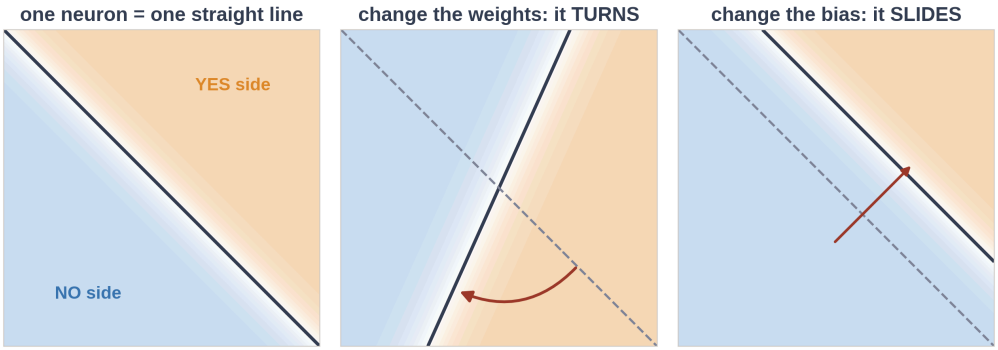
IN WORDS **how sure = squash(score)**

WITH NUMBERS **0.71 = squash(0.9)**

A very negative score gives nearly 0 ("no"); a very positive one nearly 1 ("yes"); a score of 0 gives exactly 0.5, a coin toss. You never need to compute it by hand: the shape is what matters.

> **KEY IDEA**
> One neuron with this S-curve **is** Week 2's logistic regression. Same weighted sum, same squash.

3.4 One neuron draws one straight line



Put a neuron on a map of dots. Wherever the score z is positive it says "yes", and wherever z is negative it says "no". The border between the two, where $z = 0$, is a **straight line**. Changing the weights **turns** the line; changing the bias **slides** it. That is all one neuron can ever draw. On the two moons, the best straight line gets about 87% of new dots right, and the mistakes sit exactly where the crescents curl into each other.

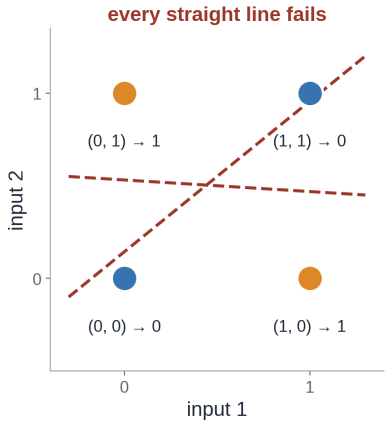
★ KEEP THIS

- A **neuron** multiplies each input by a **weight**, adds them up with a **bias**, then bends the total with an **activation**.
- With a sigmoid squash, one neuron is logistic regression (Week 2).
- One neuron can only draw **one straight line**.

Part 4 Guess, with a network: how it is wired

One neuron draws one line. This part shows what a **team** of neurons can do, using the smallest possible example: four dots.

4.1 Four dots one neuron cannot sort

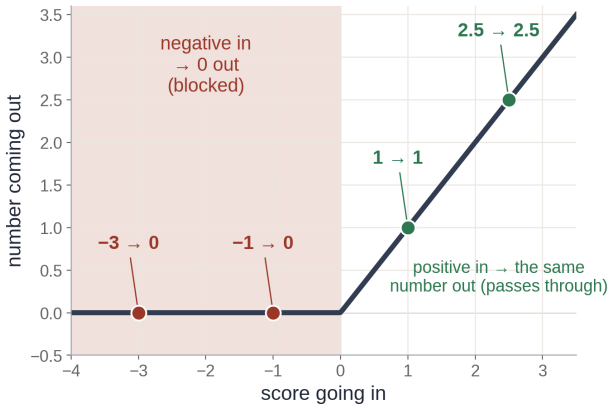


Two switches, each off (0) or on (1). Colour a dot orange if **exactly one** switch is on, blue if both are off or both are on. This is called **XOR** (“exclusive or”).

Try to draw one straight line with both orange dots on one side and both blue dots on the other. You cannot; nobody can. So **one neuron can never learn XOR.** When this was pointed out in 1969, it nearly stopped neural-network research for a decade.

4.2 ReLU: the bend used inside networks

Before we wire the fix, meet the bend that modern networks use inside, between layers. It is the simplest rule in this booklet.



THE FORMULA $\mathrm{ReLU}(z) = \max(0, z)$

IN WORDS **output = the score if it is positive, otherwise 0**

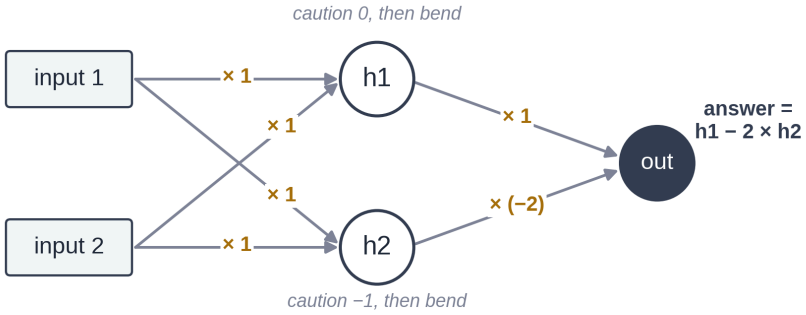
WITH NUMBERS **ReLU(−3) = 0 · ReLU(0.5) = 0.5 · ReLU(2) = 2**

EVERYDAY ANALOGY

A one-way valve: positive numbers flow through unchanged; negative numbers are stopped at zero.

Why would anyone want that? Because it puts a **corner** in a straight line. A neuron followed by ReLU is switched **off** for some inputs and **on** for others, and that switching is what lets a network bend its boundary. The name stands for “rectified linear unit”: “rectified” means the negative half is cut off.

4.3 Wiring two hidden neurons



bend = ReLU: a negative number becomes 0, anything else passes through

Put two neurons in the middle. Each does the umbrella sum on the two switches and then the ReLU bend. The second has a caution of −1, so it only fires when **both** switches are on. A final neuron combines them.

THE FORMULA $h_1 = \mathrm{ReLU}(x_1 + x_2) \quad h_2 = \mathrm{ReLU}(x_1 + x_2 - 1) \quad \hat{y} = h_1 - 2h_2$

IN WORDS **hidden 1 = bend(switch 1 + switch 2) · hidden 2 = bend(switch 1 + switch 2 − 1)**
answer = hidden 1 − 2 × hidden 2

WITH NUMBERS **dot (1, 1): 2 = ReLU(1 + 1) · 1 = ReLU(1 + 1 − 1) · answer 0 = 2 − 2 × 1**

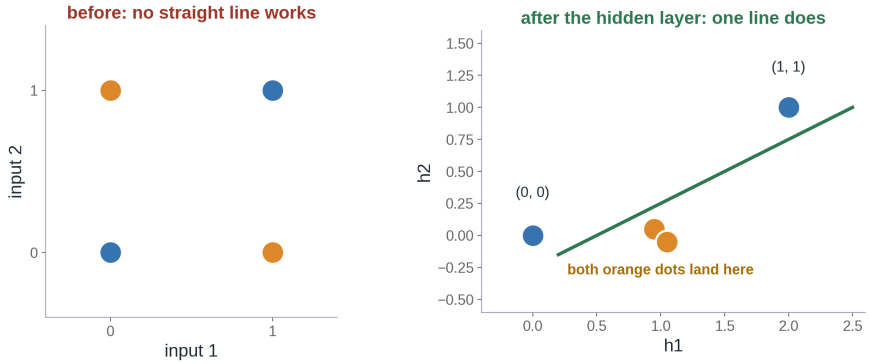
Worked example: all four dots.

dot	h_1 = ReLU($x_1 + x_2$)	h_2 = ReLU($x_1 + x_2 - 1$)	answer = $h_1 - 2h_2$
(0, 0)	0	ReLU(−1) = 0	0
(0, 1)	1	0	1
(1, 0)	1	0	1
(1, 1)	2	1	0

Exactly XOR: 0, 1, 1, 0.

Any hidden layer can be written in one line, whatever its size: $\boldsymbol{h} = \mathrm{ReLU}(W\boldsymbol{x} + \boldsymbol{b})$. In words: every neuron in the layer does its own weighted sum of the **same** inputs, then bends. W is a table holding every neuron's weights (one row per neuron) and $\boldsymbol{b}$ holds their cautions.

4.4 The hidden layer moves the dots

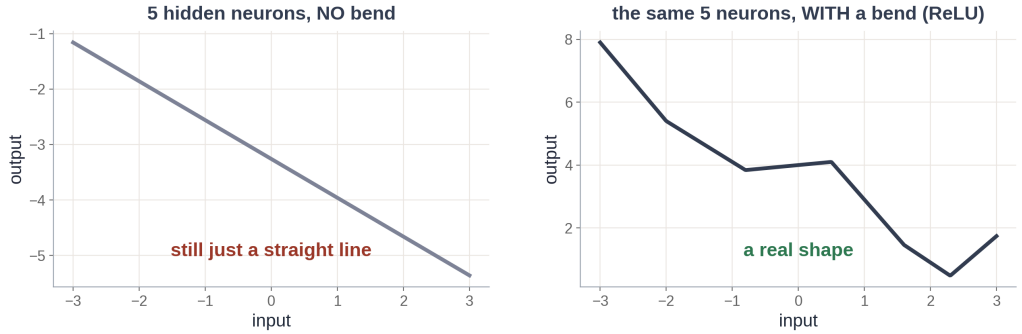


Here is **why** it works. Plot each dot by its new numbers (h_1, h_2) instead of its switches. (0, 0) stays at (0, 0). (0, 1) and (1, 0) **both** land on (1, 0). (1, 1) moves to (2, 1). The two orange dots now sit on top of each other, and one straight green line separates them from the blues.

KEY IDEA

A hidden layer moves the dots so that a straight line works. The output never sees the original inputs, only the new numbers the hidden layer made. That is the whole idea of a neural network.

4.5 Why every neuron needs a bend

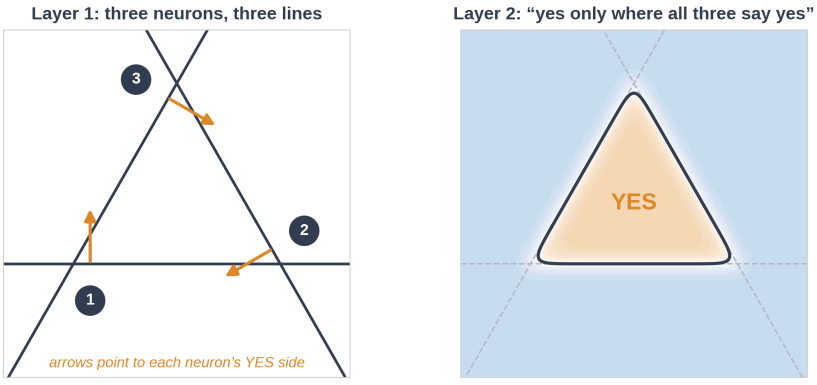


Join two straight rulers end to end without a hinge and you still have a straight line; join a hundred and it is still straight. Layers without a bend behave the same way: two straight-line layers in a row multiply out into one straight-line layer, $W_2(W_1x) = (W_2W_1)x$. Left: five neurons with no bend give a straight line whatever their weights. Right: the same five neurons with ReLU make a real shape. **Without the bend, a deep network is no better than one neuron.**

GOING FURTHER · OPTIONAL · WHICH BENDS EXIST

Sigmoid (0 to 1) and tanh (−1 to 1) are smooth “dimmer switches”; their flat ends make learning stall in deep networks (Part 7′s telephone game). ReLU has no flat end on its positive side, which is why it became the default inside networks. Sigmoid is still used at the very end of a yes/no network, to turn the final score into “how sure”.

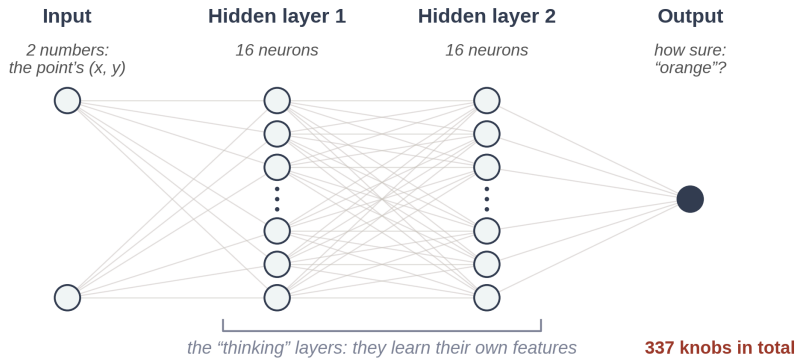
4.6 From four dots to a real network



Three neurons draw three lines; one more neuron says “yes” only where all three agree.

EVERYDAY ANALOGY

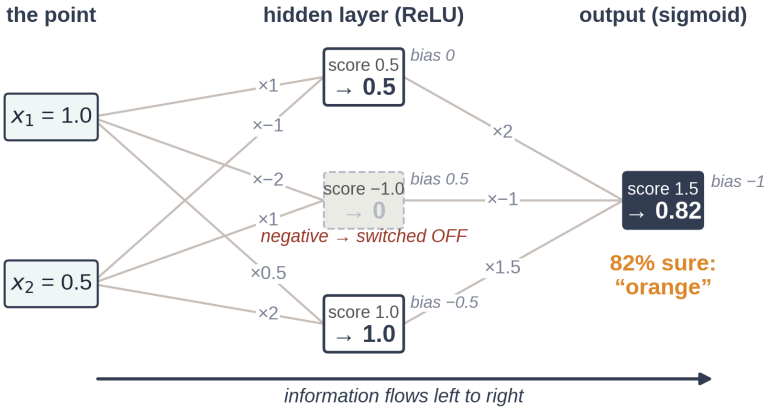
Think of an office. The front desk passes in the facts (the inputs). Every analyst (a hidden neuron) reads all of them and writes one note. The manager (the output) reads every note and decides. Every arrow between them is a dial.



Today's network takes two numbers in (a dot's position on the map), has two hidden layers of 16 neurons, and gives one answer out: how sure it is that the dot is orange. We write it 2 → 16 → 16 → 1.

> **Worked example: counting the dials.** Layer 1: 2 inputs × 16 neurons = 32 weights, plus 16 biases: 48. Layer 2: 16 × 16 = 256 weights, plus 16 biases: 272. Output: 16 weights plus 1 bias: 17. Total: 48 + 272 + 17 = **337 dials**. Learning means finding good values for all 337 at once.

4.7 Asking the network a question: the forward pass

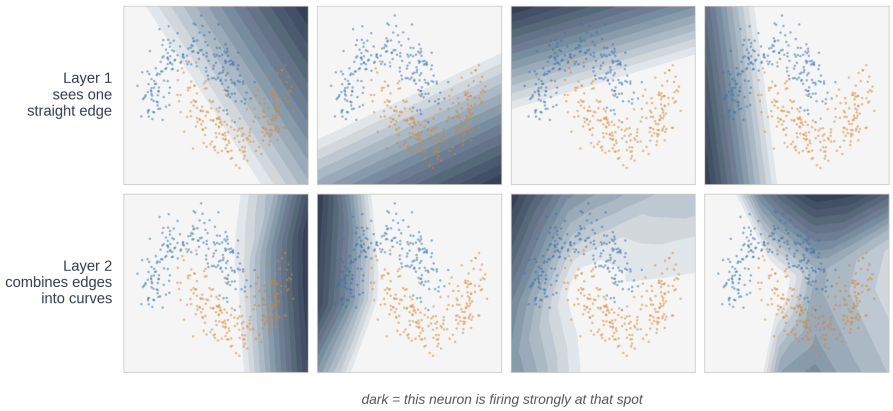


Asking a network something is called the **forward pass**: numbers flow from left to right, layer by layer.

> **Worked example: a tiny network.** The dot is at (1.0, 0.5). Top hidden neuron: 1.0 × 1 + 0.5 × (−1) + 0 = 0.5, and ReLU keeps 0.5. Middle: 1.0 × (−2) + 0.5 × 1 + 0.5 = −1.0, and ReLU turns it into 0: this neuron is switched off for this dot. Bottom: 1.0 × 0.5 + 0.5 × 2 − 0.5 = 1.0. Output: 0.5 × 2 + 0 × (−1) + 1.0 × 1.5 − 1 = 1.5, which the sigmoid squashes to 0.82. **82% sure the dot is orange.**

For our real network the whole forward pass is one formula, read from the inside out:

THE FORMULA	$\hat{y} = \sigma(W_3\,\mathrm{ReLU}(W_2\,\mathrm{ReLU}(W_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2) + b_3)$
IN WORDS	**how sure = squash(sum of bends(sum of bends(sum of the dot)))**
WITH NUMBERS	**a dot → 16 numbers → 16 numbers → how sure it is orange**



What real hidden neurons respond to. Layer 1 neurons each react to one straight edge; layer 2 neurons combine edges into curved pieces. Nobody told the network to look for these.

This links straight back to Week 5: PCA found new axes in the data; a hidden layer finds new features, made for the job at hand.

★ **KEEP THIS**

- **XOR**: four dots no straight line can separate, so one neuron can never learn it.
- **ReLU(z) = max(0, z)**: a negative score becomes 0, a positive one passes through.
- A **hidden layer** moves the dots into a new arrangement where a straight line works.
- Without a **bend**, any number of layers collapses into one straight line.
- Today's network, 2 → 16 → 16 → 1, has **337 dials**. The **forward pass** runs it left to right.

Part 5 Check: how wrong was it?

A network gives answers. Before it can improve, it needs one number that says how wrong they are. That number is the **loss**.



EVERYDAY ANALOGY

Like a darts score. Each dart is one guess; the bullseye is the right answer. Close together in the middle: a small loss. Scattered: a big loss. Lower is better, and **learning means making this one number small**.

5.1 For number answers: the average squared miss



THE FORMULA

$$L = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

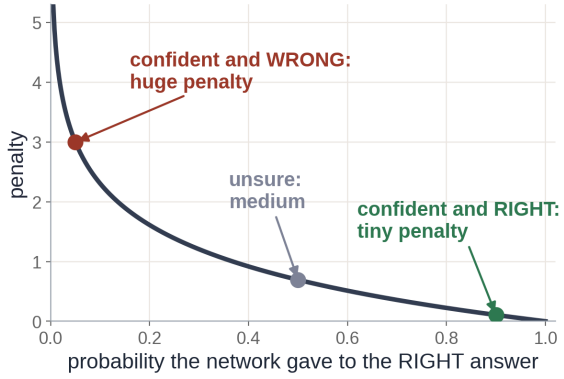
IN WORDS **loss = average over all the houses of (guess − real price)²**

WITH NUMBERS **8,700 = [(120 − 180)² + (150 − 240)² + (180 − 300)²] ÷ 3**

$\sum$ (“sigma”)	add up. The little $i = 1$ and n say “for every house, from the first to the last”
n	how many examples: here 3 houses
$\hat{y}_i$, y_i	the guess and the real price for house number i

Squaring does two jobs: it makes every miss positive (so misses above and below the line cannot cancel out), and it makes **big misses count much more** than small ones. A miss twice as big costs four times as much.

5.2 For yes/no answers: cross-entropy



For a yes/no answer we ask: how sure was the network of the **right** answer?

- 90% sure and right → penalty 0.1
- 50/50 → penalty 0.7
- only 5% on the right answer → penalty 3.0

EVERYDAY ANALOGY

A pub quiz where you bet points on how sure you are. Hedging costs a little; betting everything on the wrong answer costs a lot.

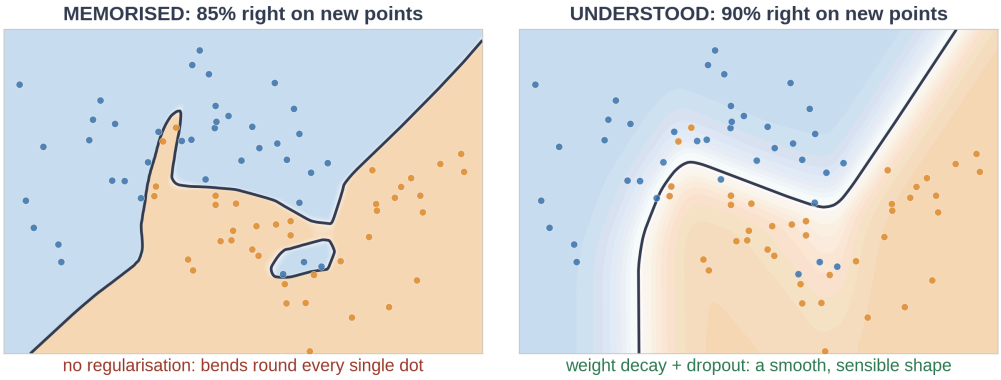
THE FORMULA $$L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

IN WORDS **penalty = − log(how sure it was of the right answer)**

WITH NUMBERS **0.1 = − log(0.9)** **3.0 = − log(0.05)**

Here y is the truth (1 for orange, 0 for blue) and $\hat{y}$ is how sure the network is that the dot is orange. Because y is either 1 or 0, only one half of the formula is ever switched on: the half for the right answer. The log of a number below 1 is negative, so the minus sign turns it into a positive penalty. This is the loss our moons network uses; Week 8 explains why it is the natural choice.

5.3 Memorising is not learning



The loss that matters is the loss on dots the network has **never seen**. Here a big network learns from only 80 noisy dots. Left: it bends round every single practice dot, gets all of them right, and only 85% of new dots. Right: the same network with the brakes from Part 7 draws a smooth shape and gets 90%.

IN PLAIN ENGLISH

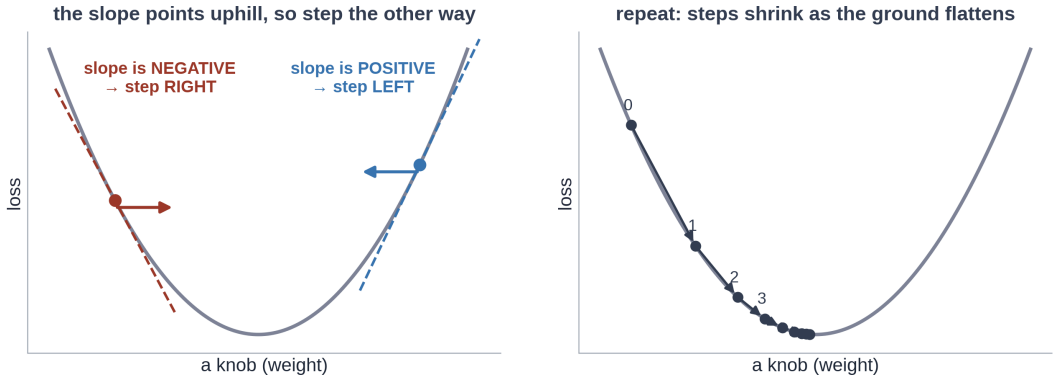
Always compare the practice score with the score on new data. A small gap means it learned the pattern; a big gap means it memorised the examples, like a student who learned last year's answers by heart. This is Week 2's overfitting, and the test set's whole purpose.

★ **KEEP THIS**

- The **loss** is one number for how wrong the guesses are; learning means making it small.
- Number answers: the **average squared miss**. Yes/no answers: **cross-entropy**, where confident and wrong costs most.
- Judge a model on data it has **never seen**. A big gap between practice and test scores means **memorising**.

Part 6 Nudge: walking downhill

6.1 Feel the slope, take a step



Picture one dial and the loss for every setting of it: a valley. You cannot see the whole valley, but at any point you can feel the **slope**. If the ground goes down to the right, step right; if it goes up to the right, step left. Always step against the uphill. Near the bottom the ground flattens, so the steps **shrink on their own**: nobody tells the walker to slow down. This method is called **gradient descent**.

6.2 The formula of the day

THE FORMULA

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}$$

IN WORDS

new dial = old dial − step size × slope

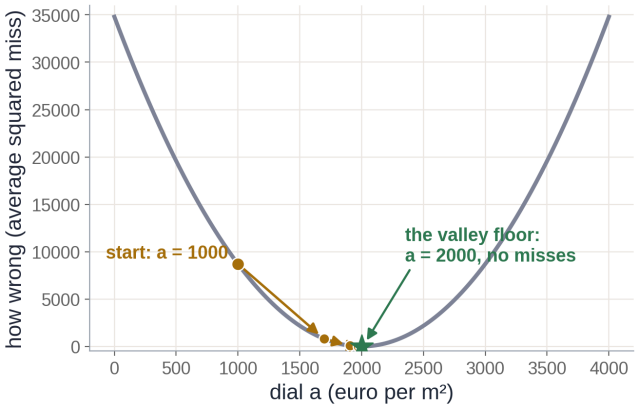
WITH NUMBERS

1,696 = 1,000 − 40 × (−17.4)

θ (“theta”)	any dial
$\frac{\partial L}{\partial \theta}$ (“dee-L by dee-theta”)	the **slope** (also called the **gradient**): how much the loss changes if you turn this dial a little
η (“eta”)	the **step size**, also called the **learning rate**: a hyperparameter you choose
$\leftarrow$	“becomes”

The minus sign means “go downhill”. If the slope is negative (turning the dial up lowers the loss), minus a negative moves the dial **up**.

6.3 The house dial finds its own way down

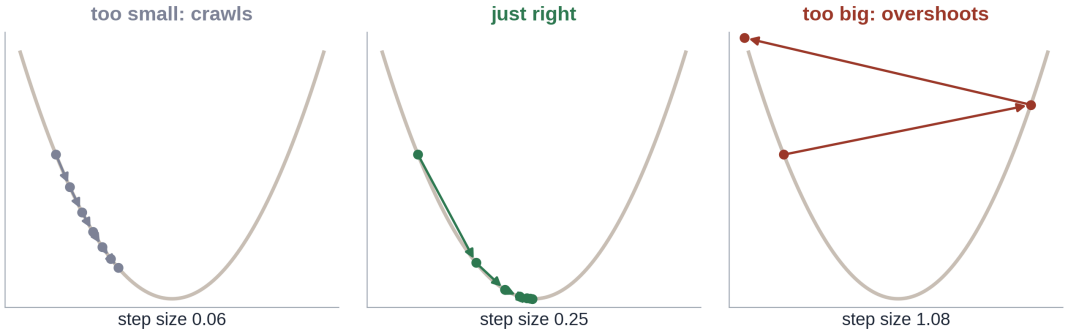


step	a	slope	loss
0	1000.0	−17.40	8700.0
1	1696.0	−5.29	804.0
2	1907.6	−1.61	74.3
3	1971.9	−0.49	6.9
4	1991.5	−0.15	0.6
5	1997.4	−0.05	0.1

> **Worked example: the first step.** Start at $a = 1000$ with a step size of 40. The slope there is −17.4, so the new dial is 1000 − 40 × (−17.4) = 1000 + 696 = 1696. Repeat: the loss falls from 8700 to 0.1 in five steps, and the dial arrives at 2000 **without anyone telling it the answer**.

> GOING FURTHER · OPTIONAL · WHERE THE SLOPE COMES FROM
>
> For the house loss, the slope with respect to a is the average of 2 × (miss) × (size ÷ 1000) over the three houses. At $a = 1000$ the misses are −60, −90 and −120, giving (2 × (−60) × 0.06 + 2 × (−90) × 0.09 + 2 × (−120) × 0.12) ÷ 3 = −17.4.

6.4 The step size is the whole ballgame



Too small: tiny shuffles; correct, but you run out of time. **Just right:** down in a few steps. **Too big:** the walker leaps over the valley, lands higher on the other side, and leaps back further still. On the house dial, step sizes up to about 115 settle; above that every step overshoots more than the last and the dial flies off. On our real moons network, after 200 rounds, a step of 0.003 is still crawling (84% right), 0.05 is smooth (97%), and 3 never settles (48%, a coin toss). Far too big and the program prints `nan` (“not a number”) with no error message.

> ⋆ KEEP THIS
>
> - **Gradient descent**: feel the slope, take a small step downhill, repeat.
> - **new dial = old dial − step size × slope** ($\theta \leftarrow \theta - \eta\, \partial L/\partial\theta$).
> - The **step size** (learning rate) is a hyperparameter. Too small: crawls. Too big: bounces or explodes. If training misbehaves, check it first.

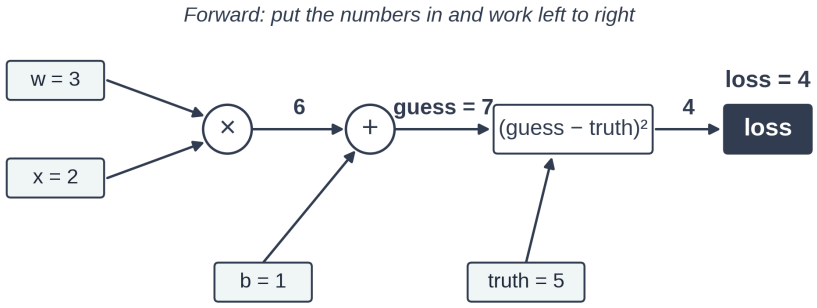
Part 7 Blame: passing it backwards

Walking downhill needs the slope for **every** dial: 337 of them for our network. This part shows how to get them all at once. It is the heart of the week, so it goes slowly, with one dial worked by hand.

EVERYDAY ANALOGY

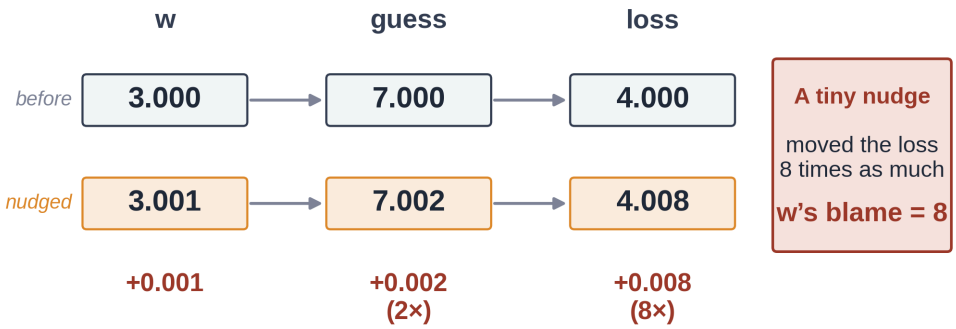
A football team has just lost 3–0. The coach does not sack everyone: she replays the goals backwards, from the last mistake to the first, and gives each player a share of the blame. Each player then changes that much.

7.1 One dial, worked by hand



One dial $w = 3$, one input $x = 2$, a bias $b = 1$, and a true answer of 5. The guess is 3 × 2 + 1 = 7, the miss is 7 − 5 = 2, and the loss is 2² = 4. Should w go up or down?

7.2 Blame by nudging



The simplest idea in the world: turn the dial a tiny bit and watch. Moving w from 3.000 to 3.001 moves the guess from 7.000 to 7.002 (twice the nudge) and the loss from 4.000 to 4.008 (eight times the nudge). So w's **blame** is 8. It is positive, which means turning w up makes things worse: turn it **down**.

THE FORMULA

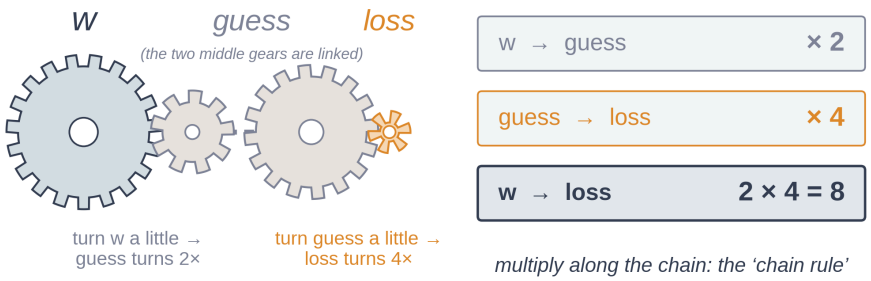
$$\frac{\partial L}{\partial w} \approx \frac{L(w+\varepsilon) - L(w)}{\varepsilon}$$

IN WORDS **blame of w = (loss after a tiny nudge − loss before) ÷ size of the nudge**

WITH NUMBERS **8 = (4.008 − 4.000) ÷ 0.001**

Here ε ("epsilon") is the tiny nudge. A dial's blame is exactly the slope gradient descent needs. The trouble is cost: each nudge needs a whole extra run of the network, so 337 dials would need 337 extra runs for **every single step**, and big networks have billions of dials.

7.3 The chain rule: blame multiplies, like gears



Look at the nudge again. Turning w moved the guess 2× (because $x = 2$). Turning the guess moved the loss 4× (because the miss was 2, and the slope of a square at a gap of 2 is 2 × 2). So turning w moved the loss 2 × 4 = 8×: the same 8 we found by nudging. Effects along a chain **multiply**, like gears. This is the **chain rule**.

THE FORMULA

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w}$$

IN WORDS

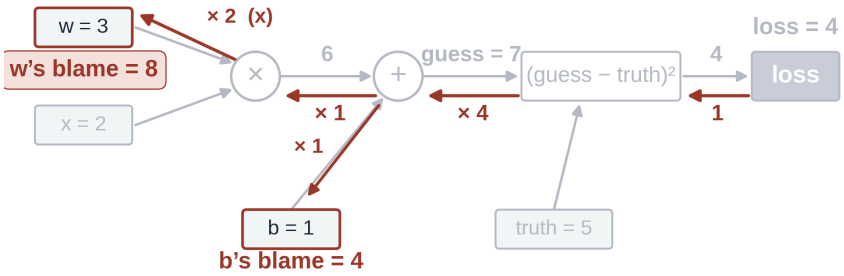
(how much w moves the loss) = (how much the guess moves the loss) × (how much w moves the guess)

WITH NUMBERS

8 = 4 × 2

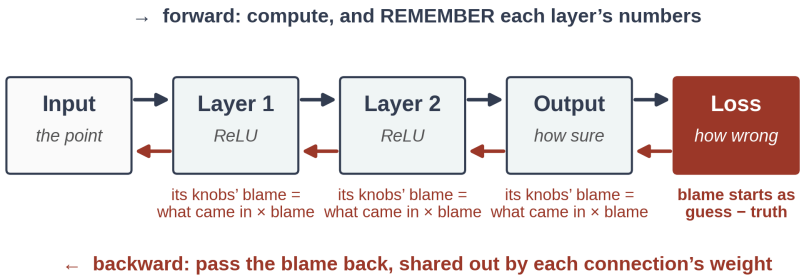
7.4 Backpropagation: walk back from the loss

Backward: start at the loss with 1, and multiply as you walk right to left



> **Worked example: walking backwards.** Start with 1 at the loss. Through the square: × 4, so the guess carries blame 4. Through the +: adding passes blame straight through (× 1), so b’s blame is 4. Through the ×: multiply by $x = 2$, so w’s blame is 8. **One walk back, and every dial has its blame.** One step downhill with step size 0.01 would then set w = 3 − 0.01 × 8 = 2.92, giving a guess of 6.84 and a loss of about 3.39: it learned.

7.5 In a real network: three rules

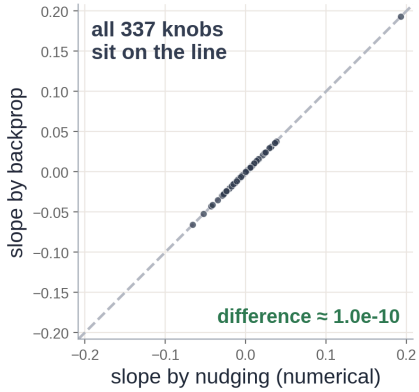


Going forward, every layer works out its numbers and **remembers** them. Going back, the blame starts at the end and each layer passes its share to the layer before. The whole of backpropagation is three rules:

$\delta_{\text{out}} = \hat{y} - y$	**blame at the end = guess − truth**
$\dfrac{\partial L}{\partial W} = \boldsymbol{h}^T \delta$	**blame of a weight = what came in × blame going out** (the "× x" step above)
$\delta_{\text{back}} = (\delta W^T) \odot \text{ReLU}'(z)$	**blame handed back = blame × the same weights, and nothing through a switched-off neuron**

δ ("delta") means blame. $\text{ReLU}'(z)$ is 1 if the neuron fired and 0 if ReLU switched it off, so a neuron that said nothing on the way forward gets no blame on the way back.

7.6 Checking the homework



How do we know backpropagation is right? Do both: blame by nudging (slow but sure) and blame by backprop (fast), for all 337 dials, and compare. Every dot sits on the diagonal: they agree to about one part in ten billion. This is called a **gradient check**, and you will run one in the lab.

PyTorch's `loss.backward()` does exactly this backwards walk for you, for any network you write.

★ **KEEP THIS**

- A dial's **blame** (its gradient) is how much the loss moves when you nudge that dial.
- **Backpropagation** finds every dial's blame in one walk backwards from the loss, multiplying along the chain (**the chain rule**).
- Nudging is slow but sure: use it to **check** backprop (a gradient check).

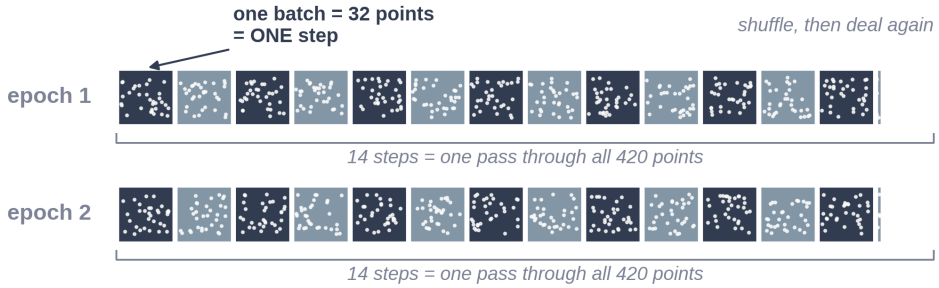
Part 8 The loop, running

8.1 The loop as four formulas

1 Guess	$\hat{y} = f(x; \theta)$	run the network with today's dials θ
2 Check	$L = \text{loss}(\hat{y}, y)$	one number for how wrong
3 Blame	$\frac{\partial L}{\partial \theta}$	every dial's blame, by walking back
4 Nudge	$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}$	every dial a small step downhill

In PyTorch these four rows are `model(X)`, `loss_fn(pred, Y)`, `loss.backward()` and `optimiser.step()`, plus `optimiser.zero_grad()` to wipe the previous round's blame. In the lab you write each one yourself in NumPy first, so none of them is magic.

8.2 A spoonful, not the whole pot

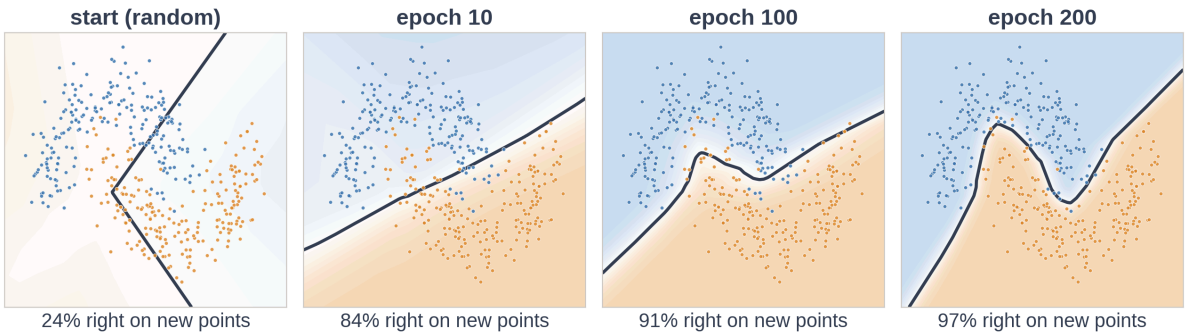


EVERYDAY ANALOGY

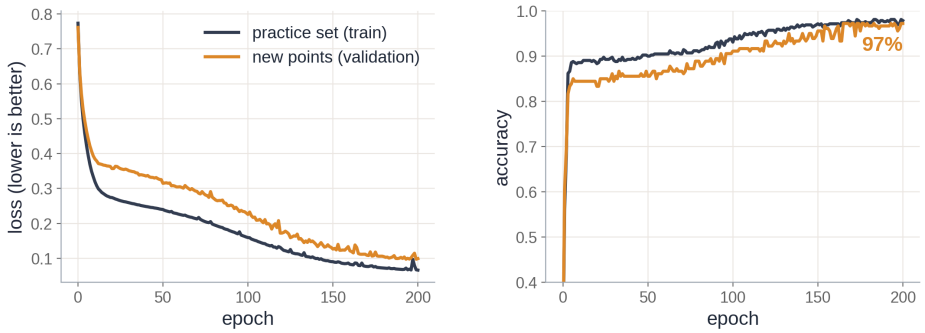
You taste a spoonful of soup, not the whole pot.

Instead of scoring all 420 practice dots before every step, we shuffle them and use 32 at a time. One spoonful is a **batch** and gives one step. After 14 batches every dot has been used once: one **epoch**. Then we shuffle and deal again. Each step is quicker, so we take many more of them, and the small randomness in each spoonful helps jiggle the network out of bad spots. Our run took 14 steps × 200 epochs = 2,800 steps.

8.3 Watching it learn

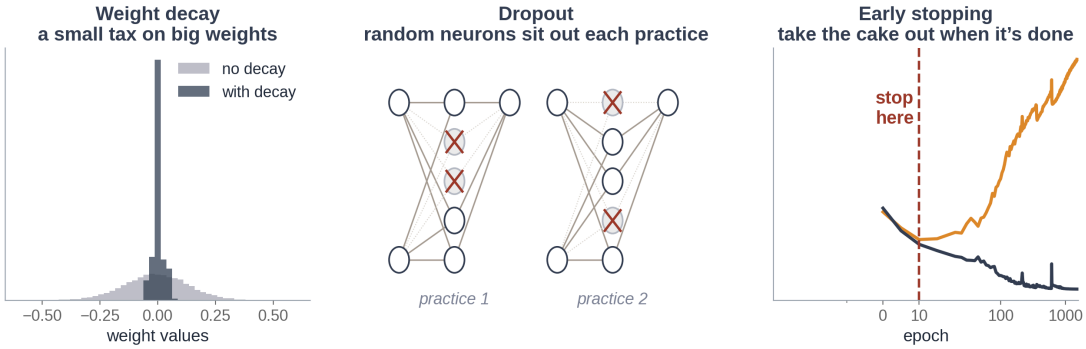


The same network at four moments. At the start, with random dials, it gets 24% of new dots right. After 10 epochs it has found a straight-ish line, like logistic regression (84%). By epoch 100 it is starting to bend (91%), and by epoch 200 it curves round both moons (97%). Nobody drew that curve; 2,800 small nudges did.



A healthy run: the loss on practice dots and on new dots falls together and levels off.

8.4 Three cures for memorising



- **Weight decay**: a small tax on big dials, so the network prefers a smooth, simple shape. This is Week 2's ridge penalty again.
- **Dropout**: in every practice round, some neurons are switched off at random, so no single neuron can be relied on and the knowledge spreads out. All neurons are back on when the network is used for real.
- **Early stopping**: take the cake out when it is done, not when the timer rings. Keep the network from the moment it did best on new data.

THE FORMULA

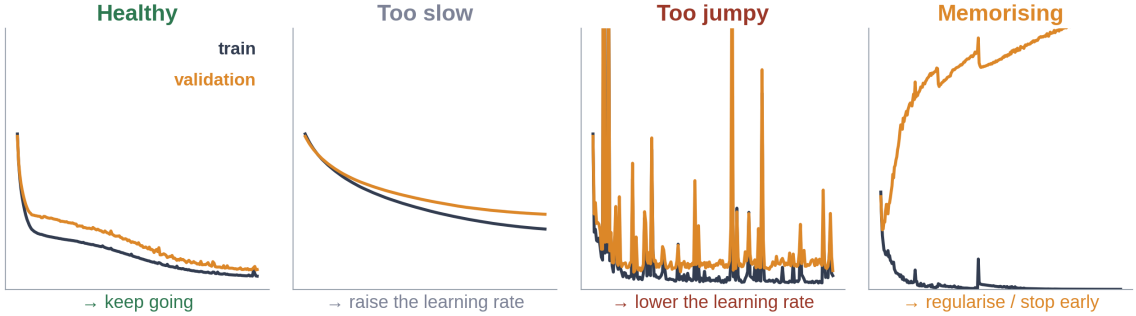
$$L_{\text{total}} = L + \frac{\lambda}{2}\sum w^2$$

IN WORDS **total loss = how wrong + tax × (every weight squared, added up)**

WITH NUMBERS **tax λ = 0.01 in our run: practice score 100% → 93%, new dots 85% → 90%**

Here λ ("lambda") is how heavy the tax is, a hyperparameter. The tax adds a little of each weight to its own blame, so every step shrinks every weight slightly towards zero: that is why it is called **decay**.

8.5 Reading the curves like a doctor



The curves look like	Diagnosis	What to change
both fall smoothly, then level off together	healthy	keep going
barely moving	too slow	a bigger step size
spiky, jumping up	too jumpy	a smaller step size
practice falls while new data rises	memorising	a cure, or stop early

IN PLAIN ENGLISH

Change **one** thing at a time, and write down what it did to the curves. That habit is the whole skill of training networks.

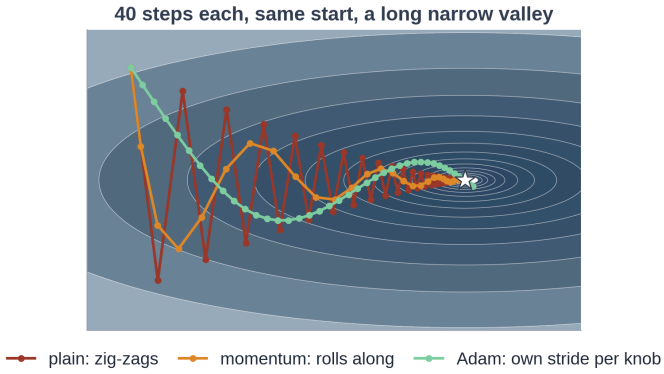
★ KEEP THIS

- Training = the four-step loop on small **batches**, epoch after epoch.
- Always draw the practice **and** test curves, and read them.
- Cures for memorising: **weight decay, dropout, early stopping**.

Part 9 Going further (optional)

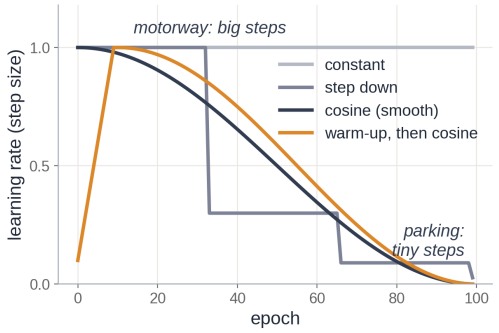
Nothing in this part is needed for the lab. Each section explains, in plain words, an idea you will meet in books and later weeks.

9.1 Smarter steps: momentum and Adam



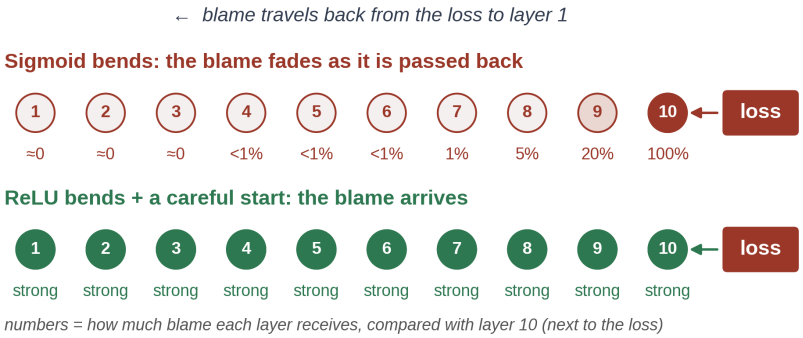
Real loss landscapes often have long, narrow valleys: steep walls and a gentle floor. Plain gradient descent points at the nearest wall, so it bounces from side to side and crawls along the floor. **Momentum** is a heavy ball rolling downhill: each step keeps some of the last step's speed, so the side-to-side wobbles cancel out and speed builds along the floor. **Adam** gives every dial its own stride, by dividing each dial's average slope by how bumpy that dial's slope usually is. Adam is the usual first choice in practice.

9.2 Big steps first, small steps later



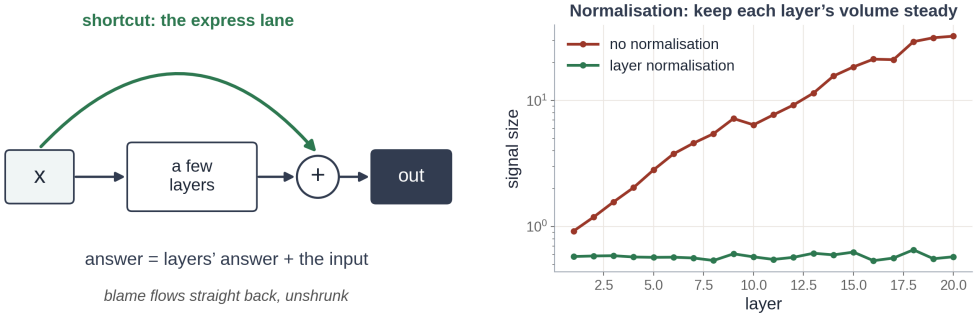
Driving into town you go fast on the motorway and slowly when you park. Training is the same: big steps early to cover ground, tiny steps late to settle into the bottom without overshooting. Shrinking the step size over time is called a **schedule**; a short **warm-up** at the start stops the first, random steps from flinging the dials too far.

9.3 The telephone game: when blame fades



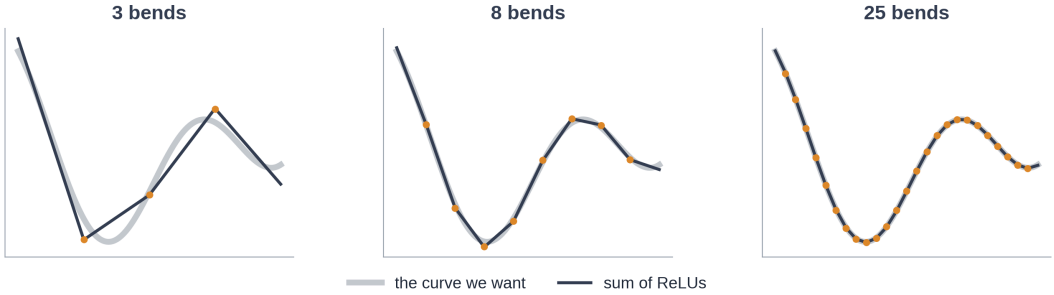
Backprop passes blame through every layer, multiplying by each bend's slope on the way. The sigmoid's slope is at most 0.25, so after ten sigmoid layers the blame reaching layer 1 is tiny, and the early layers stop learning: this is the **vanishing gradient**. ReLU has a slope of exactly 1 wherever it is switched on, and a careful random start (**He initialisation**, which scales the starting dials to the size of each layer) keeps the message strong all the way back.

9.4 Two tricks that let networks grow very deep



Shortcuts (skip connections) add a block's input straight onto its output, so the blame has an express lane home and does not fade. **Normalisation** rescales each layer's numbers to a steady size, like keeping every speaker at a comfortable volume. Transformers in Week 10 use both in every block.

9.5 Enough bends can draw any curve



Each ReLU adds one bend. With enough bends you can copy any smooth curve, like drawing a circle with many short straight strokes. This is the **universal approximation theorem**: with enough neurons, even one hidden layer can copy any smooth curve as closely as you like. The catch is that "enough" can be enormous; stacking layers reuses bends, so deep networks need far fewer neurons for the same shape.

9.6 A short history: why did this wait until 2012?

The first artificial neuron was proposed in the 1940s, and Rosenblatt's **perceptron** (1958) caused great excitement, until it was shown in 1969 that a single neuron cannot learn XOR. Stacking neurons was the known fix, but nobody could train the hidden layers until **backpropagation** was popularised in 1986. Even then, deep networks only took off around 2012, when three ingredients finally arrived together: **data** (ImageNet, 2009: 14 million labelled pictures), **compute** (graphics cards turn out to be ideal for millions of small multiply-and-adds) and **small fixes** (ReLU, better starting dials, dropout). The recipe was old; the ingredients arrived late.

Part 10 In the lab

The lab does on your own machine what this booklet does on paper, in the same order. You will:

1. Try the house dials and compute the loss (Part 2 of this booklet).
2. Build one neuron, then see one straight line fail on the moons (Part 3).
3. Wire XOR by hand with ReLU, then build the 2 → 16 → 16 → 1 network's forward pass (Part 4).
4. Write the two losses, and walk the house dial downhill with different step sizes (Parts 5 and 6).
5. Find blame by nudging, write backpropagation, and check it with a gradient check (Part 7).
6. Train the moons network, then train the same kind of network on the Week 1 credit data and compare it with Week 2′s logistic regression (Part 8).

Then, in the exercises, you fix a deliberately broken training run, one change at a time. The from-scratch backpropagation you write is the one your final project asks for, so do it carefully now and reuse it.

Practice questions

Try each question before reading the answer below it.

1. **Name the four steps of the training loop, in order, and say what each does.**

Guess (the forward pass: make a prediction), check (the loss: one number for how wrong), blame (backpropagation: each dial's share of the mistake), nudge (gradient descent: a small step downhill for every dial).

2. **Is the step size a parameter or a hyperparameter? Why?**

A hyperparameter: you choose it before training; the computer never changes it.

3. **With $\hat{y} = ax + b$, $a = 1500$ and $b = 50$ (in €k), what is the guess for a 100 m² house?**

$1500 \times 100 \div 1000 + 50 = €200k$.

4. **A neuron has inputs (2, 1), weights (0.5, −1) and bias 0.5. What is its score z?**

$2 \times 0.5 + 1 \times (-1) + 0.5 = 0.5$.

5. **What is ReLU(−2)? ReLU(3)?**

0 and 3.

6. **Why can one neuron never learn XOR?**

It can only draw one straight line, and no straight line separates the two orange dots from the two blue ones.

7. **What does a hidden layer do, in one sentence?**

It moves the dots into a new arrangement (new numbers) where a straight line does work.

8. **Why does every neuron need a bend?**

Without one, any number of layers multiplies out into a single straight-line layer.

9. **Misses of 1, −2 and 3. What is the average squared miss?**

$(1 + 4 + 9) \div 3 \approx 4.67$.

10. **Which costs more under cross-entropy: being 50/50, or being 95% sure of the wrong answer?**

95% sure of the wrong answer: about 3.0 against 0.7.

11. **Old dial 10, slope 2, step size 0.5. What is the new dial?**

$10 - 0.5 \times 2 = 9$.

12. The loss bounces up and down and never settles. What do you change first?

Make the step size smaller.

13. Nudging a dial by 0.01 raises the loss from 2.00 to 2.05. What is its blame, and which way should it turn?

0.05 ÷ 0.01 = 5: positive, so turn the dial down.

14. State the chain rule in words.

How much w moves the loss = how much the guess moves the loss × how much w moves the guess.

15. In the one-dial example, why is b’s blame 4 and w’s blame 8?

Both start from the blame of 4 on the guess. Adding b passes it through unchanged (× 1); multiplying by x = 2 doubles it for w.

16. Why do we check backpropagation by nudging, and why not use nudging all the time?

Nudging is simple and almost certainly right, so it catches bugs. But it needs one extra run per dial, which is far too slow for training.

17. 420 practice dots, batches of 32. How many steps make one epoch?

14 (the last batch is smaller).

18. Practice score 100%, test score 85%. What is happening, and name two cures.

Memorising (overfitting). Any two of weight decay, dropout, early stopping.

19. What does weight decay add to the loss, in words?

A small tax on big weights: λ/2 × the sum of every weight squared.

20. Why did deep learning take off around 2012 rather than 1986?

Data (large labelled sets), compute (graphics cards) and small fixes (ReLU, better starting dials) finally arrived together.

Formula cheat sheet

Every formula from this week, in symbols and in words.

Idea	Symbols	Words
Model	$\hat{y} = ax + b$	price = a × size + base price
Neuron	$z = w_1x_1 + w_2x_2 + w_3x_3 + b,\ \ \hat{y} = \sigma(z)$	score = sum of (trust × clue) + caution; how sure = squash(score)
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$	squash any score into 0 to 1
ReLU	$\mathrm{ReLU}(z) = \max(0, z)$	the score if positive, otherwise 0
Layer	$\boldsymbol{h} = \mathrm{ReLU}(W\boldsymbol{x} + \boldsymbol{b})$	every neuron’s weighted sum of the same inputs, then bend
Network	$\hat{y} = \sigma(W_3\ \mathrm{ReLU}(W_2\ \mathrm{ReLU}(W_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2) + b_3)$	read inside out: the forward pass
Squared miss	$L = \frac{1}{n}\sum(\hat{y}_i - y_i)^2$	average of (guess − truth)²
Cross-entropy	$L = -[y\log\hat{y} + (1-y)\log(1-\hat{y})]$	− log(how sure of the right answer)
Gradient descent	$\theta \leftarrow \theta - \eta\,\partial L/\partial\theta$	new dial = old dial − step size × slope
Blame by nudging	$\partial L/\partial w \approx (L(w+\varepsilon) - L(w))/\varepsilon$	change in loss ÷ size of the nudge
Chain rule	$\partial L/\partial w = (\partial L/\partial\hat{y}) \cdot (\partial\hat{y}/\partial w)$	the “how much it moves” numbers multiply
Backprop	$\delta_{\text{out}} = \hat{y} - y;\ \ \partial L/\partial W = \boldsymbol{h}^T\delta$	blame at the end = guess − truth; a weight’s blame = what came in × blame going out
Weight decay	$L_{\text{total}} = L + \frac{\lambda}{2}\sum w^2$	how wrong + a tax on big weights

Glossary

activation (bend)	what a neuron does to its score after adding up: ReLU inside a network, sigmoid at the end of a yes/no network
backpropagation	finding every dial's blame in one walk backwards from the loss, using the chain rule
batch	a handful of examples (here 32) used for one step
bias	a dial that nudges a neuron's score up or down whatever the inputs say; the "caution"
blame (gradient)	how much the loss moves when you nudge one dial
chain rule	effects along a chain multiply, like gears
cross-entropy	the loss for yes/no answers: − log of how sure the model was of the right answer
dial (parameter)	a number inside the model that learning adjusts: a weight or a bias
dropout	switching random neurons off during training so that none becomes indispensable
early stopping	keeping the model from its best moment on new data
epoch	one pass through all the training data
forward pass	asking the network a question: numbers flow from input to output
gradient check	comparing backprop's blame with blame by nudging, to prove the code is right
gradient descent	feel the slope, step downhill, repeat
hidden layer	a layer between input and output; it makes new numbers the next layer can use
hyperparameter	a setting you choose before training: step size, number of layers, epochs
learning rate (step size)	how far each dial moves in one step
loss	one number for how wrong the guesses are; lower is better
neuron	weigh the inputs, add them up with a bias, then bend
overfitting (memorising)	learning the practice examples instead of the pattern: great in practice, worse on new data
ReLU	the bend max(0, z): a negative score becomes 0, a positive one passes through
sigmoid	the S-curve that squashes any score into 0 to 1
weight	a dial that says how much one input counts; negative means it argues against
weight decay	a small tax on big weights that keeps the model smooth
XOR	four dots, "one or the other but not both", that no single straight line can separate